

RTL-to-GDSII Implementation of a UART Transceiver Using OpenLane2

Technical Project Report

RTL Design, Verification, Physical Design, and Signoff
using the OpenLane2 Open-Source ASIC Flow

Author

Shivam Chaurasiya

B.Tech in Electronics and Communication Engineering

Project Type

Self-Directed Learning Project

Development Environment

Language	Verilog HDL
Technology	SKY130A PDK
RTL Verification	Verilator
Logic Synthesis	Yosys
Physical Design	OpenLane2 / OpenROAD
Physical Verification	Magic, KLayout, Netgen
Operating System	Ubuntu 24.04 (WSL2)
Final Deliverable	GDSII Layout

GitHub Repository

<https://github.com/shivam-du/uart-rtl-to-gdsii>

July 2026

Abstract

This project presents the complete RTL-to-GDSII implementation of a Universal Asynchronous Receiver-Transmitter (UART) Intellectual Property (IP) core using the open-source OpenLane2 ASIC design flow and the SkyWater SKY130 Process Design Kit (PDK). The objective of the project is to demonstrate the complete digital ASIC design methodology, beginning from Register Transfer Level (RTL) design and ending with a manufacturable GDSII layout.

The UART IP core was developed in Verilog HDL using a modular architecture consisting of a Baud Rate Generator, UART Transmitter, UART Receiver, and a top-level integration module. Functional verification of each module was performed using dedicated Verilog testbenches, and the simulation waveforms confirmed the correct operation of the complete UART communication system.

The verified RTL was synthesized using Yosys and implemented through the OpenLane2 automated RTL-to-GDSII flow. The physical design process included floorplanning, standard-cell placement, clock tree synthesis (CTS), global routing, and detailed routing using OpenROAD. Post-layout verification was carried out using OpenRCX for parasitic extraction, OpenSTA for timing analysis, Magic for Design Rule Checking (DRC), and Netgen for Layout Versus Schematic (LVS) verification. Power integrity was evaluated through IR-drop analysis, while the final manufacturability assessment confirmed successful completion of antenna, DRC, and LVS verification.

The implementation achieved successful timing closure, efficient placement and routing, zero routing overflow, negligible IR drop, zero DRC violations, successful LVS matching, and complete manufacturability verification. The final output of the project is a verified GDSII layout of the UART IP core, demonstrating the effectiveness of the OpenLane2 open-source ASIC design flow. This project provides practical experience with modern digital ASIC implementation and highlights the complete transformation of a synthesizable RTL design into a fabrication-ready integrated circuit.

Keywords: UART, ASIC, RTL-to-GDSII, Verilog HDL, OpenLane2, OpenROAD, SKY130 PDK, Physical Design, Static Timing Analysis, Magic DRC, Netgen LVS, GDSII.

Contents

Abstract	1
1 Introduction	1
1.1 Introduction	1
1.2 UART Communication Protocol	1
1.3 Applications of UART	2
1.4 Project Overview	3
1.5 Motivation	3
1.6 Objectives	3
1.7 Design Specifications	4
1.8 Project Methodology	4
2 System Architecture	6
2.1 UART Transceiver Architecture	6
2.2 Internal Architecture	7
2.3 Module Description	7
2.4 Baud Rate Generator	7
2.5 UART Transmitter	9
2.5.1 Finite State Machine of UART Transmitter	10
2.6 UART Receiver	10
2.6.1 Finite State Machine of UART Receiver	11
2.7 Data Transmission and Reception Flow	12
2.7.1 UART Transmission Flow	12
2.7.2 UART Reception Flow	13
2.8 Top Module Integration	13
3 RTL Design and Verification	15
3.1 RTL Module Organization	15
3.2 Baud Rate Generator	15
3.3 UART Transmitter	16
3.4 UART Receiver	16
3.5 UART Top Module	18
3.6 Functional Verification	18
4 RTL-to-GDSII Design Flow Using OpenLane2	20
4.1 OpenLane2 Design Flow	20
4.2 Project Directory Structure	22
4.3 Design Configuration	22
4.4 RTL Linting Using Verilator	22

4.5	Logic Synthesis Using Yosys	23
4.6	Complete OpenLane2 Implementation Pipeline	24
5	Physical Design Implementation	25
5.1	Floorplanning	26
5.2	Global and Detailed Placement	27
5.3	Clock Tree Synthesis	28
5.4	Global and Detailed Routing	30
5.5	Physical Design Summary	31
6	Physical Verification and Signoff	32
6.1	Verification Flow	32
6.2	RC Extraction	33
6.3	Static Timing Analysis	33
6.4	IR Drop Analysis	33
6.5	Design Rule Checking	34
6.6	Layout Versus Schematic Verification	34
6.7	Final GDSII Layout	34
6.8	Manufacturability Assessment	34
7	Results and Discussion	36
7.1	Functional Verification	36
7.1.1	Baud Rate Generator	36
7.1.2	UART Transmitter	36
7.1.3	UART Receiver	36
7.1.4	UART Top Module	37
7.2	RTL Synthesis Results	37
7.3	Floorplanning Results	38
7.4	Placement Results	38
7.5	Clock Tree Synthesis Results	39
7.6	Routing Results	39
7.7	Power Integrity Results	39
7.8	Physical Verification Results	40
7.9	Overall Discussion	40
8	Conclusion	41

List of Figures

1.1	Standard UART Communication Frame.	2
1.2	Complete RTL-to-GDSII ASIC Design Flow adopted in this project. . . .	5
2.1	Top-Level Architecture of the UART Transceiver.	6
2.2	Internal Architecture of the UART Transceiver.	7
2.3	Architecture of the Baud Rate Generator.	8
2.4	Architecture of the UART Transmitter.	9
2.5	Finite State Machine of the UART Transmitter.	10
2.6	Finite State Machine of the UART Receiver.	11
2.7	Flowchart of UART Data Transmission.	12
2.8	Flowchart of UART Data Reception.	13
2.9	Block Diagram of the UART Top Module.	14
3.1	RTL schematic of the Baud Rate Generator generated using Yosys. . . .	16
3.2	RTL schematic of the UART transmitter generated using Yosys.	17
3.3	RTL schematic of the UART receiver generated using Yosys.	18
3.4	RTL hierarchy of the complete UART top module generated using Yosys.	18
4.1	Complete RTL-to-GDSII implementation flow using OpenLane2.	21
4.2	Logic synthesis flow performed using Yosys.	23
5.1	Physical design implementation flow using OpenROAD.	25
5.2	Generated floorplan of the UART ASIC design.	26
5.3	Standard-cell placement generated by OpenROAD.	27
5.4	Clock Tree Synthesis (CTS) stages.	29
5.5	Final routed layout generated by OpenROAD.	30
6.1	Physical verification and signoff flow.	32
6.2	Final GDSII layout of the UART IP core viewed in KLayout.	35
7.1	Simulation waveform of the Baud Rate Generator.	36
7.2	Simulation waveform of the UART transmitter.	37
7.3	Simulation waveform of the UART receiver.	37
7.4	Simulation waveform of the complete UART top module.	37

List of Tables

1.1	Design Specifications	4
2.1	Functional Description of UART Modules	8
3.1	UART RTL modules	15
3.2	Baud Generator ports	16
3.3	UART transmitter ports	16
3.4	UART receiver ports	17
4.1	Major stages of the OpenLane2 ASIC design flow	20
4.2	Important OpenLane2 configuration parameters	23
4.3	Open-source EDA tools used in this project	24
4.4	Generated outputs during RTL-to-GDSII implementation	24
5.1	Floorplanning metrics	26
5.2	Placement metrics	27
5.3	Cell distribution after detailed placement	28
5.4	Clock Tree Synthesis statistics	28
5.5	Global and detailed routing statistics	30
5.6	Routing resource utilization by metal layer	31
5.7	Summary of physical design implementation	31
6.1	Generated SPEF files	33
6.2	IR-drop analysis results	33
6.3	Magic DRC summary	34
6.4	LVS verification summary	34
6.5	Manufacturability verification summary	35
7.1	RTL synthesis summary	38
7.2	Floorplanning results	38
7.3	Placement statistics	38
7.4	CTS statistics	39
7.5	Routing statistics	39
7.6	IR-drop summary	39
7.7	Verification summary	40

Chapter 1

Introduction

1.1 Introduction

Universal Asynchronous Receiver-Transmitter (UART) is one of the most widely used serial communication protocols for exchanging digital data between electronic devices. Unlike synchronous communication protocols, UART operates without a shared clock signal between the transmitter and receiver. Instead, both devices communicate using a preconfigured baud rate, allowing reliable asynchronous data transmission over a pair of communication lines.

Due to its simple hardware architecture, low implementation cost, and reliable operation, UART has become a standard communication interface in microcontrollers, embedded systems, System-on-Chip (SoC) designs, FPGA prototypes, industrial automation, and Internet of Things (IoT) devices. It is commonly used for communication between processors, sensors, development boards, GPS modules, Bluetooth modules, and various peripheral devices.

In modern digital integrated circuits, UART is generally implemented as a reusable Intellectual Property (IP) core. Such IP cores are integrated into larger digital systems to provide serial communication while minimizing silicon area, power consumption, and hardware complexity. Consequently, implementing a UART IP core provides an excellent platform for understanding the complete digital ASIC design methodology from RTL development to physical layout generation.

1.2 UART Communication Protocol

UART performs full-duplex serial communication using two independent communication lines: the Transmit (TX) line and the Receive (RX) line. Data is transmitted serially by converting parallel input data into a sequence of bits at the transmitter, while the receiver reconstructs the original parallel data by sampling the incoming serial stream using the configured baud rate.

A standard UART frame consists of four fundamental fields:

- **Start Bit** – Indicates the beginning of a data frame.
- **Data Bits** – Typically 8 bits representing the transmitted data.
- **Optional Parity Bit** – Used for simple error detection.

- **Stop Bit(s)** – Indicates the completion of the transmitted frame.

The transmitter and receiver must be configured with the same communication parameters, including baud rate, number of data bits, parity configuration, and stop bits, to ensure reliable communication.

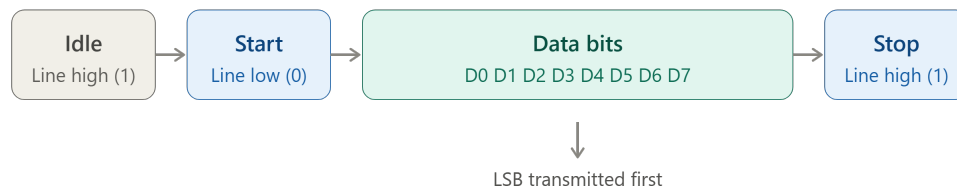


Figure 1.1: Standard UART Communication Frame.

Figure 1.1 illustrates the structure of a standard UART data frame. Communication begins with a start bit followed by the data bits, an optional parity bit, and one or more stop bits. This frame structure enables asynchronous communication without requiring a dedicated clock signal between communicating devices.

1.3 Applications of UART

Because of its simplicity and reliability, UART is widely employed in both academic and industrial electronic systems. Some common applications include:

- Embedded systems and microcontrollers
- FPGA and ASIC-based digital systems
- Sensor interfacing
- GPS and Bluetooth communication modules
- Industrial automation and control systems
- Internet of Things (IoT) devices
- Debugging and serial terminal communication
- System-on-Chip (SoC) communication interfaces

The widespread adoption of UART makes it one of the most important communication peripherals in digital integrated circuits. Consequently, implementing a UART Transceiver IP Core using a complete RTL-to-GDSII ASIC design flow provides valuable practical experience in modern digital VLSI design methodologies.

1.4 Project Overview

This project focuses on the complete RTL-to-GDSII implementation of a UART Transceiver IP Core using the OpenLane2 open-source ASIC design flow. The UART transceiver was designed in Verilog HDL and functionally verified using Verilator before proceeding through the complete physical implementation flow.

The implementation includes RTL design, functional verification, logic synthesis, floorplanning, power distribution network generation, placement, clock tree synthesis (CTS), global and detailed routing, parasitic extraction, static timing analysis (STA), IR-drop analysis, Design Rule Checking (DRC), Layout Versus Schematic (LVS) verification, and final GDSII layout generation. Throughout the implementation, reports generated at each stage were analyzed to evaluate design quality, area utilization, routing efficiency, timing performance, and manufacturability.

The project demonstrates how an RTL design is transformed into a fabrication-ready integrated circuit using an entirely open-source ASIC design environment based on the SkyWater SKY130A Process Design Kit (PDK).

1.5 Motivation

Open-source Electronic Design Automation (EDA) tools have significantly simplified digital ASIC design by providing accessible alternatives to proprietary software. OpenLane2 integrates multiple open-source tools into a complete RTL-to-GDSII flow, enabling designers and students to gain practical experience in modern ASIC implementation.

The motivation behind this project is to understand every stage of the digital ASIC design flow by implementing a real communication IP core. Instead of limiting the work to RTL simulation, this project emphasizes complete physical implementation, report analysis, timing closure, routing optimization, physical verification, and manufacturability assessment.

1.6 Objectives

The primary objectives of this project are:

- Design a synthesizable UART Transceiver IP Core using Verilog HDL.
- Perform functional verification using Verilator.
- Implement the complete RTL-to-GDSII ASIC flow using OpenLane2.
- Analyze synthesis, placement, routing, timing, and utilization reports.
- Perform RC extraction, IR-drop analysis, and static timing analysis.
- Verify the final layout using Magic DRC, KLayout DRC, and Netgen LVS.
- Generate a manufacturable GDSII layout.

1.7 Design Specifications

Table 1.1 summarizes the design environment and implementation specifications used throughout this project.

Table 1.1: Design Specifications

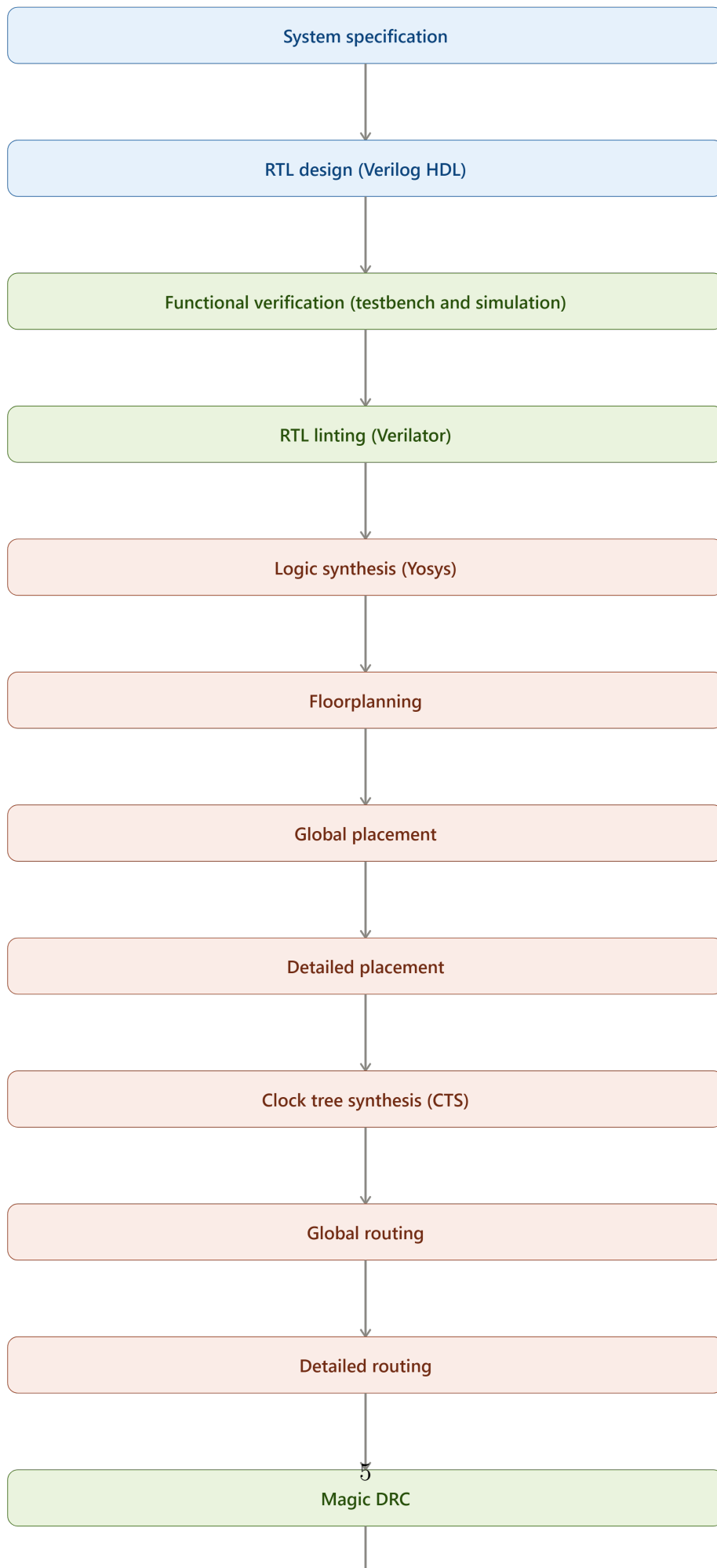
Parameter	Specification
Design	UART Transceiver IP Core
Hardware Description Language	Verilog HDL
Technology Node	130 nm CMOS
Process Design Kit	SKY130A PDK
ASIC Flow	OpenLane2
RTL Verification	Verilator
Logic Synthesis	Yosys
Physical Design	OpenROAD
Physical Verification	Magic, KLayout, Netgen
Operating System	Ubuntu 24.04 (WSL2)
Final Output	GDSII Layout

1.8 Project Methodology

The complete implementation follows a standard digital ASIC design methodology beginning with RTL design and ending with GDSII generation. Each implementation stage produces reports that are analyzed to evaluate timing performance, silicon area, routing quality, power integrity, and physical correctness.

As illustrated in Figure 1.2, the implementation begins with RTL development and functional verification, followed by logic synthesis, floorplanning, placement, clock tree synthesis, routing, RC extraction, static timing analysis, physical verification, and final GDSII layout generation. Each stage contributes to the successful realization of a fabrication-ready UART Transceiver ASIC.

The subsequent chapters discuss each implementation stage in detail, including the UART architecture, RTL implementation, verification methodology, physical design, report analysis, and signoff verification.



Chapter 2

System Architecture

2.1 UART Transceiver Architecture

The UART Transceiver developed in this project is a modular digital design consisting of four major functional blocks: the UART Top module, Baud Rate Generator, UART Transmitter, and UART Receiver. The UART Top module integrates all submodules and controls the flow of data between the transmitter and receiver while sharing a common baud clock generated by the Baud Rate Generator.

The modular architecture improves design readability, simplifies verification, and allows individual modules to be synthesized and tested independently before integration. Such a hierarchical design methodology is commonly adopted in modern ASIC design to improve scalability and reusability.

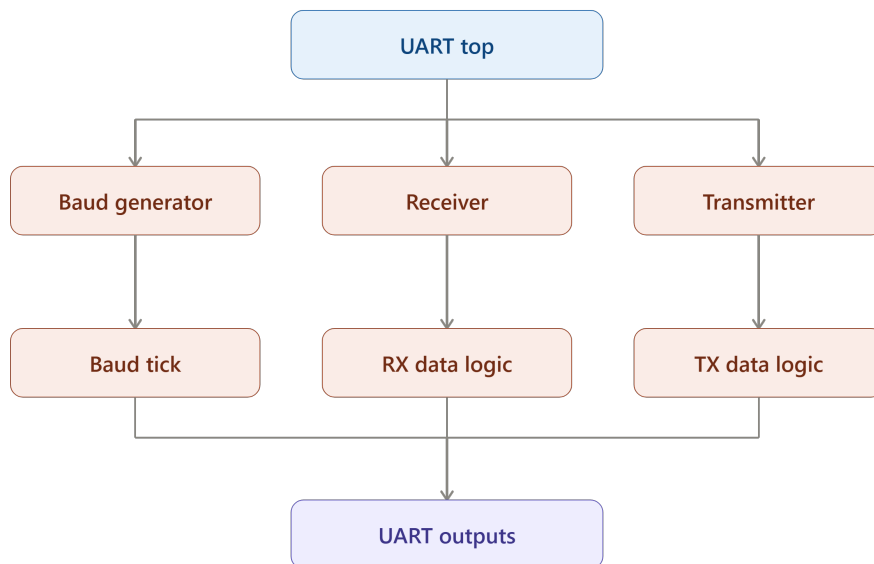


Figure 2.1: Top-Level Architecture of the UART Transceiver.

Figure 2.1 illustrates the top-level organization of the UART Transceiver. The transmitter and receiver communicate through independent serial communication paths while both modules operate using the baud clock generated by the Baud Rate Generator.

2.2 Internal Architecture

Figure 2.2 presents the complete internal architecture of the implemented UART Transceiver. The design consists of interconnected functional modules responsible for baud generation, serial transmission, serial reception, and top-level control.

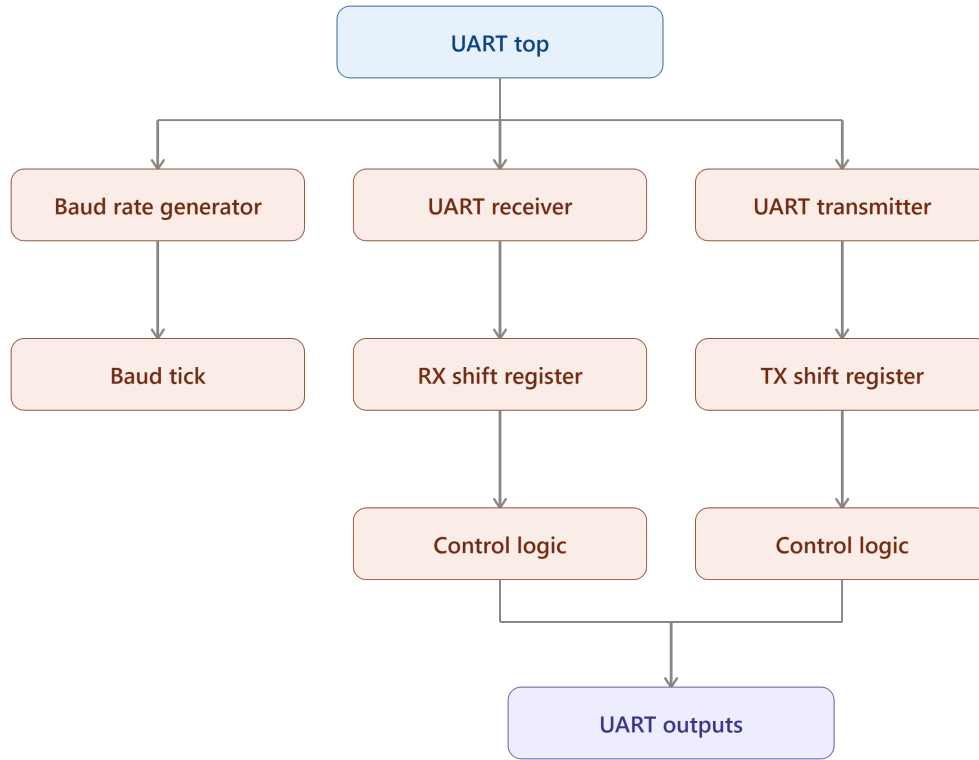


Figure 2.2: Internal Architecture of the UART Transceiver.

The hierarchical organization enables each module to perform a dedicated function while maintaining synchronized communication throughout the design. This approach simplifies debugging, verification, and future design modifications.

2.3 Module Description

Table 2.1 summarizes the functionality of each major module implemented in the UART Transceiver.

The following sections describe the architecture and operation of each module in detail.

2.4 Baud Rate Generator

The Baud Rate Generator is responsible for producing the baud clock required by both the UART transmitter and receiver. It divides the high-frequency system clock to generate timing pulses corresponding to the selected baud rate. Since both communication modules

Table 2.1: Functional Description of UART Modules

Module	Function
UART Top	Integrates all functional modules and manages overall communication.
Baud Rate Generator	Generates baud clock pulses from the system clock for synchronized data transmission and reception.
UART Transmitter	Converts parallel input data into serial format according to the UART communication protocol.
UART Receiver	Receives serial data and reconstructs the original parallel data while validating the received frame.

operate using the same baud timing, accurate baud generation is essential for reliable serial communication.

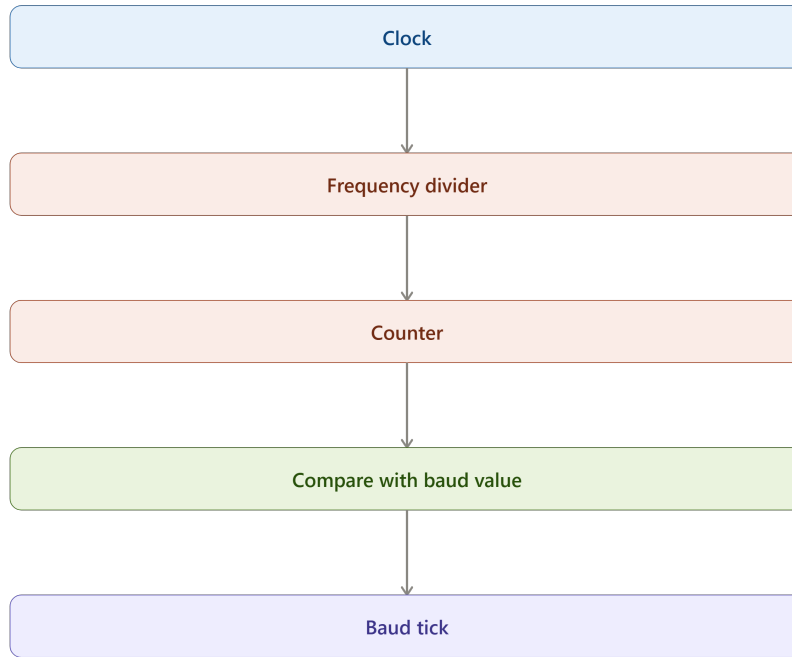


Figure 2.3: Architecture of the Baud Rate Generator.

The generated baud clock is distributed to both transmitter and receiver modules, ensuring synchronized operation throughout the communication process.

2.5 UART Transmitter

The UART Transmitter converts 8-bit parallel input data into a serial bit stream according to the UART communication protocol. Transmission begins only when the `tx_start` control signal is asserted. The transmitter first sends the start bit, followed by the data bits in Least Significant Bit (LSB) first order, and finally transmits the stop bit to complete the communication frame.

The transmitter operates synchronously with the baud clock generated by the Baud Rate Generator, ensuring accurate timing for every transmitted bit.

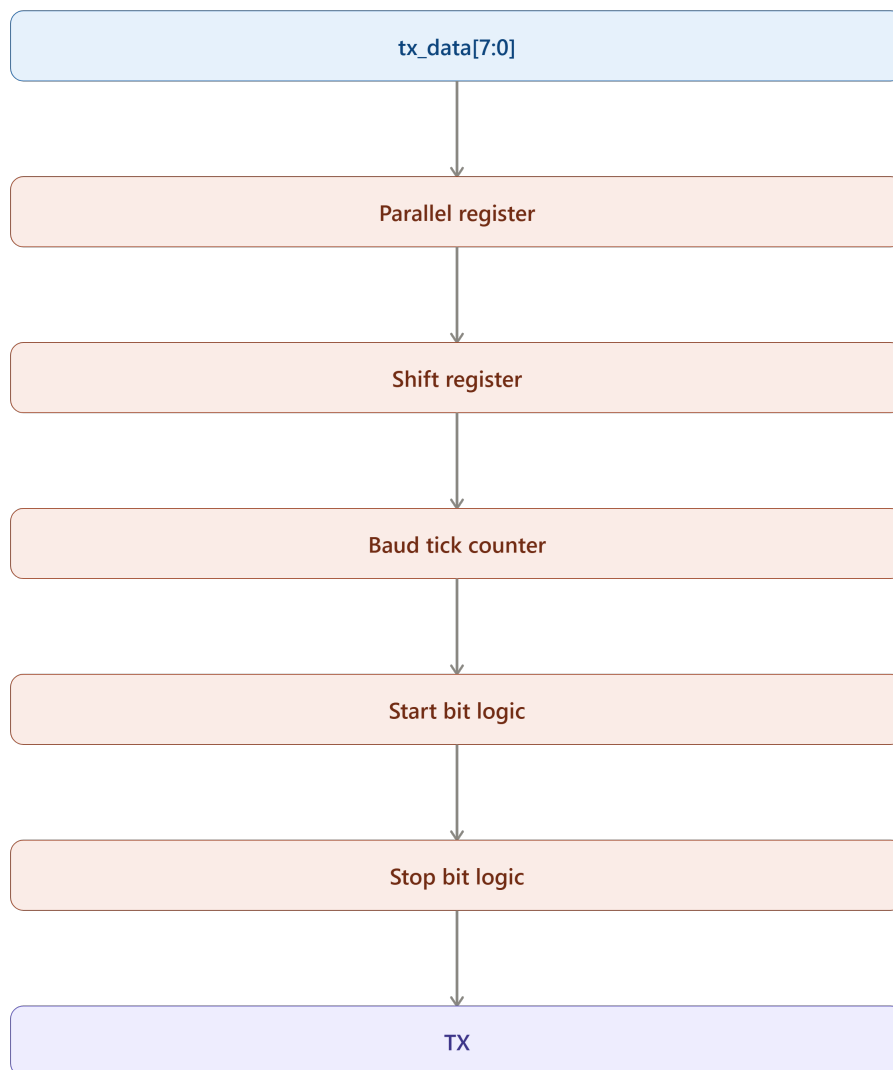


Figure 2.4: Architecture of the UART Transmitter.

Figure 2.4 illustrates the internal organization of the transmitter module. The transmitter consists of control logic, shift registers, baud clock synchronization, and output logic that serializes the input data according to the UART protocol.

The major functions performed by the transmitter are:

- Accept 8-bit parallel input data.
- Wait for the transmission enable signal.
- Generate the start bit.
- Shift data serially in LSB-first order.
- Generate the stop bit.
- Indicate transmission completion.

2.5.1 Finite State Machine of UART Transmitter

The transmitter operation is controlled by a Finite State Machine (FSM), which ensures that each communication frame follows the UART protocol correctly. The FSM sequentially controls the transmission of the start bit, data bits, and stop bit while maintaining synchronization with the baud clock.

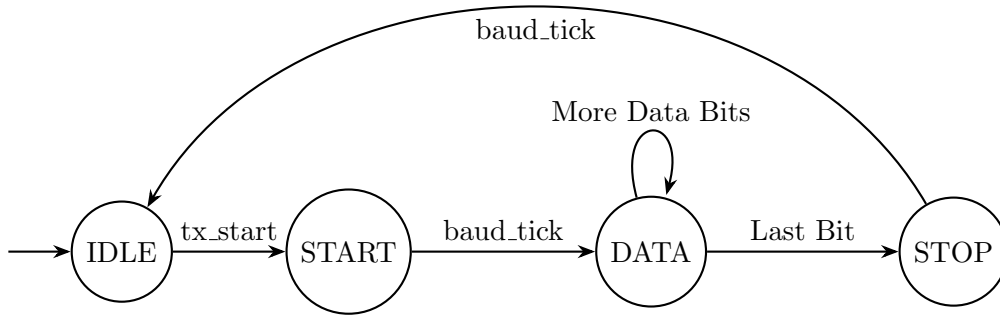


Figure 2.5: Finite State Machine of the UART Transmitter.

The transmitter FSM consists of the following operating states:

- **Idle** – Waits for the transmission request.
- **Start** – Sends the start bit.
- **Data** – Serially transmits the 8-bit data.
- **Stop** – Sends the stop bit and returns to the Idle state.

The FSM guarantees that every transmitted frame follows the required UART timing and frame structure.

2.6 UART Receiver

The UART Receiver performs the reverse operation of the transmitter by converting incoming serial data into an 8-bit parallel output. The receiver continuously monitors the RX line and begins reception after detecting the falling edge of the start bit. Data bits are sampled according to the baud clock and reconstructed into the original parallel data.

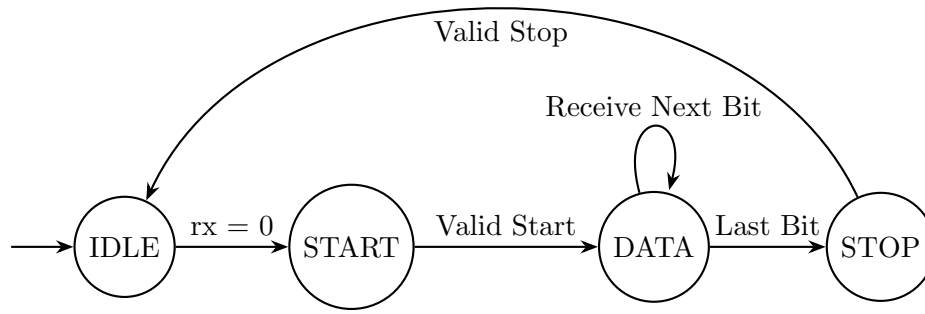


Figure 2.6: Finite State Machine of the UART Receiver.

Once all bits are successfully received, the receiver validates the stop bit before asserting the `rx_done` signal to indicate successful reception.

The receiver performs the following functions:

- Detect the incoming start bit.
- Sample serial data using the baud clock.
- Reconstruct the original 8-bit parallel data.
- Verify the stop bit.
- Assert the reception completion signal.

2.6.1 Finite State Machine of UART Receiver

Similar to the transmitter, the receiver operation is governed by a Finite State Machine (FSM). The FSM synchronizes data sampling with the baud clock and controls the complete reception process from start-bit detection to successful frame completion.

The receiver FSM consists of four primary states:

- **Idle** – Waits for the detection of the start bit.
- **Start** – Confirms the validity of the detected start bit.
- **Data** – Samples and stores the incoming serial bits.
- **Stop** – Verifies the stop bit and completes the reception process.

This state-based implementation ensures reliable data reception while minimizing synchronization errors during asynchronous communication.

2.7 Data Transmission and Reception Flow

The operation of the UART Transceiver can be divided into two independent processes: data transmission and data reception. Although both modules operate independently, they are synchronized using the baud clock generated by the Baud Rate Generator. The following flowcharts illustrate the sequence of operations performed during transmission and reception.

2.7.1 UART Transmission Flow

The transmission process begins when valid parallel data and the transmission enable signal are applied to the UART Transmitter. The transmitter serializes the data by generating the start bit, transmitting the eight data bits, and finally appending the stop bit before returning to the idle state.

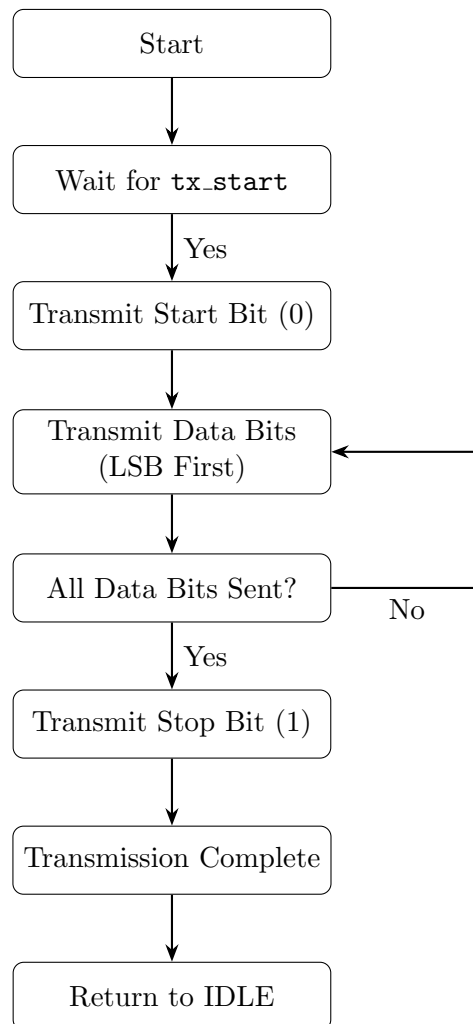


Figure 2.7: Flowchart of UART Data Transmission.

Figure 2.7 shows the sequential execution of the transmission process. Each state transition is synchronized with the baud clock to ensure accurate serial data transmission.

2.7.2 UART Reception Flow

The receiver continuously monitors the RX line for the detection of a valid start bit. Once detected, the receiver samples the incoming serial data using the baud clock, reconstructs the original 8-bit parallel data, verifies the stop bit, and asserts the reception complete signal.

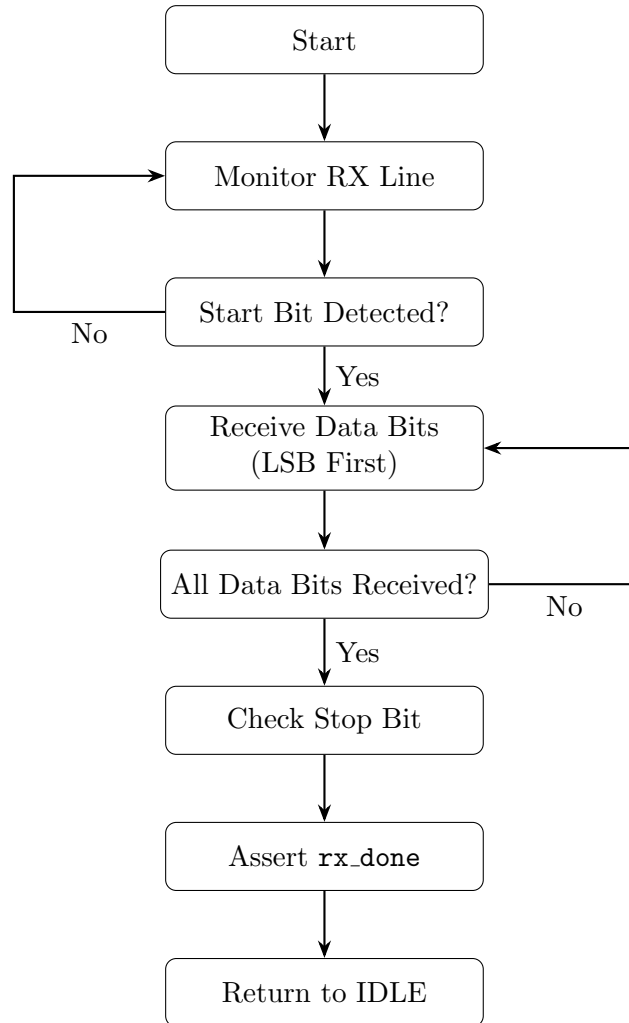


Figure 2.8: Flowchart of UART Data Reception.

Figure 2.8 illustrates the sequence followed by the receiver to recover the transmitted data while maintaining synchronization with the communication frame.

2.8 Top Module Integration

The UART Top module integrates the Baud Rate Generator, UART Transmitter, and UART Receiver into a single functional IP core. It distributes the baud clock to both communication modules, accepts parallel input data for transmission, receives serial input data, and provides the reconstructed parallel output.

Figure ?? illustrates the interaction between all major functional modules within the UART Transceiver. The top-level controller coordinates the complete communication process by connecting the baud generation, transmission, and reception blocks.

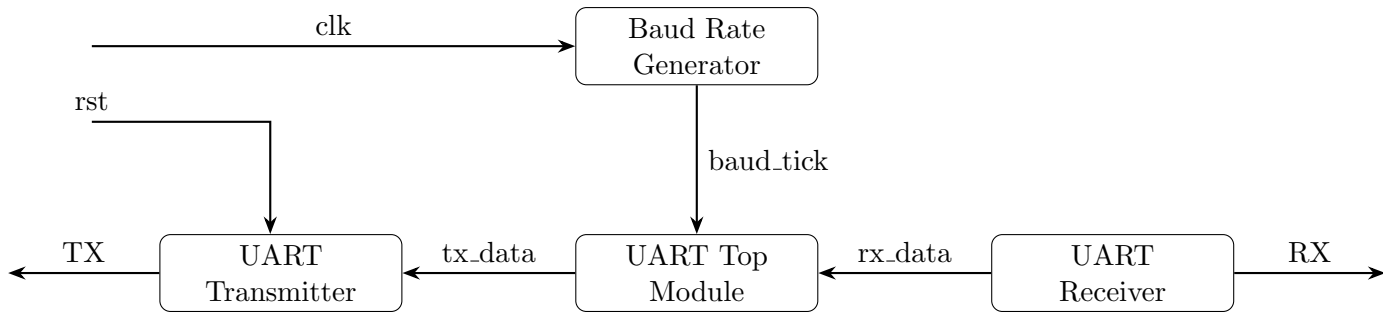


Figure 2.9: Block Diagram of the UART Top Module.

The architecture presented in this chapter forms the basis of the RTL implementation described in the next chapter. The following chapter discusses the Verilog HDL implementation of each module along with functional simulation and verification prior to ASIC synthesis.

Chapter 3

RTL Design and Verification

The UART IP core was developed using Verilog HDL following a modular design methodology. The complete RTL design consists of four independent modules: the Baud Rate Generator, UART Transmitter, UART Receiver, and the UART Top module. Each module was verified individually using dedicated testbenches before performing complete system-level verification.

The RTL implementation was written to be fully synthesizable and compatible with the SkyWater SKY130 standard cell library used by the OpenLane2 ASIC design flow.

The verification process consisted of individual module simulations followed by complete UART functionality testing.

3.1 RTL Module Organization

The complete UART design is divided into four reusable RTL modules.

Table 3.1: UART RTL modules

Module	Function
Baud Generator	Generates baud tick from system clock
UART Transmitter	Converts parallel data into serial data
UART Receiver	Converts serial data into parallel data
UART Top	Integrates all UART modules

The modular architecture improves readability, verification, scalability and future reuse of the IP.

3.2 Baud Rate Generator

The baud rate generator produces a periodic `baud_tick` signal required by both the transmitter and receiver. It divides the incoming system clock using a programmable counter so that data transmission occurs at the required baud rate.

Module Interface

The RTL schematic generated by Yosys is shown below.

Table 3.2: Baud Generator ports

Signal	Direction	Width	Description
clk	Input	1	System clock
rst	Input	1	Active-high reset
baud_tick	Output	1	Baud timing pulse

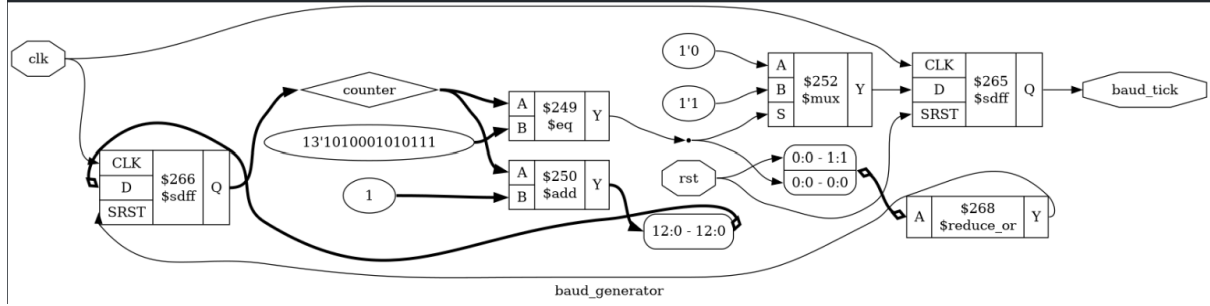


Figure 3.1: RTL schematic of the Baud Rate Generator generated using Yosys.

3.3 UART Transmitter

The transmitter converts an 8-bit parallel word into a serial UART frame consisting of a start bit, eight data bits and a stop bit. Data shifting is synchronized using the baud tick generated by the baud rate generator.

Module Interface

Table 3.3: UART transmitter ports

Signal	Direction	Width	Description
clk	Input	1	System clock
rst	Input	1	Reset
baud_tick	Input	1	Baud timing pulse
tx_start	Input	1	Transmission enable
tx_data	Input	8	Parallel input data
tx	Output	1	Serial output
tx_busy	Output	1	Busy flag

3.4 UART Receiver

The receiver continuously monitors the RX line for a valid start bit. After detecting the start bit, it samples each incoming data bit at the baud rate and reconstructs the original 8-bit data word.

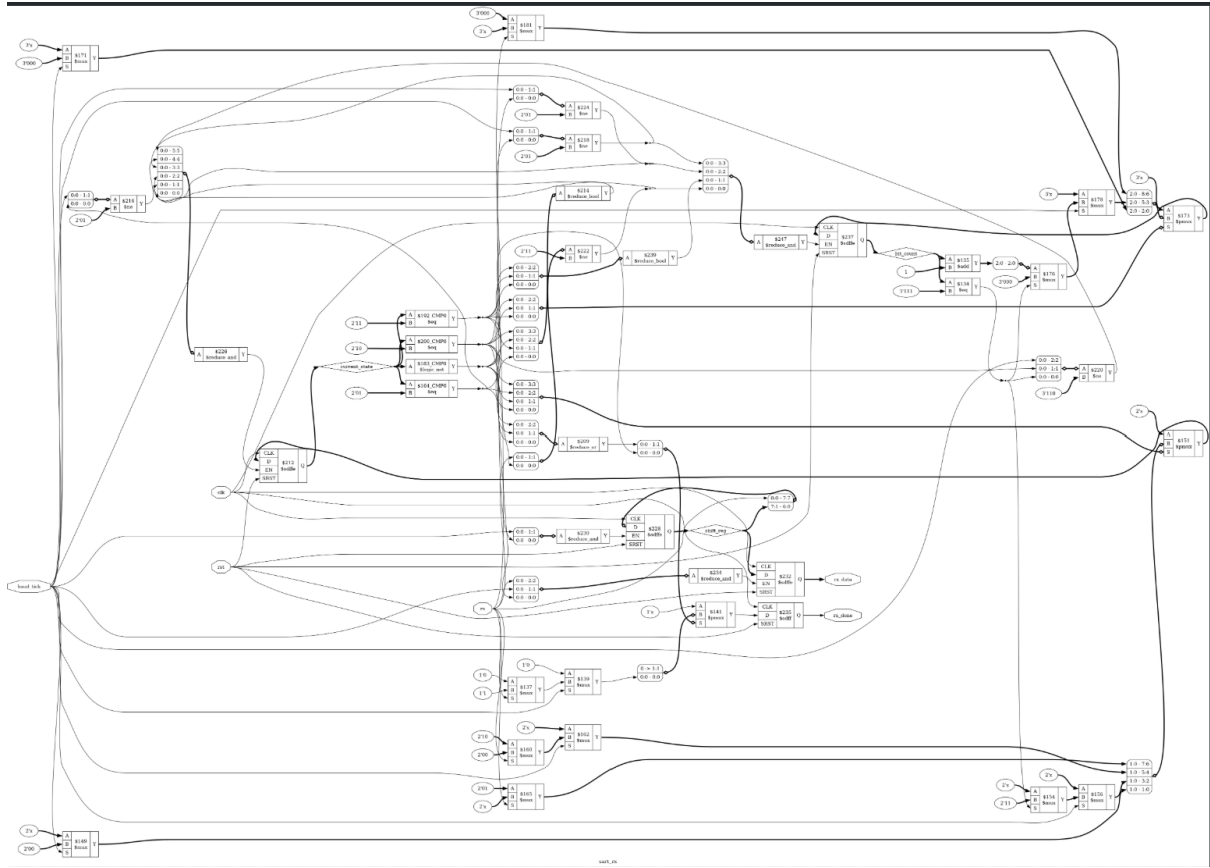


Figure 3.3: RTL schematic of the UART receiver generated using Yosys.

Module Interface

3.5 UART Top Module

The UART top module integrates the baud generator, transmitter and receiver into a complete UART IP core. The baud tick generated by the baud generator is shared by both transmitter and receiver to ensure synchronized operation.

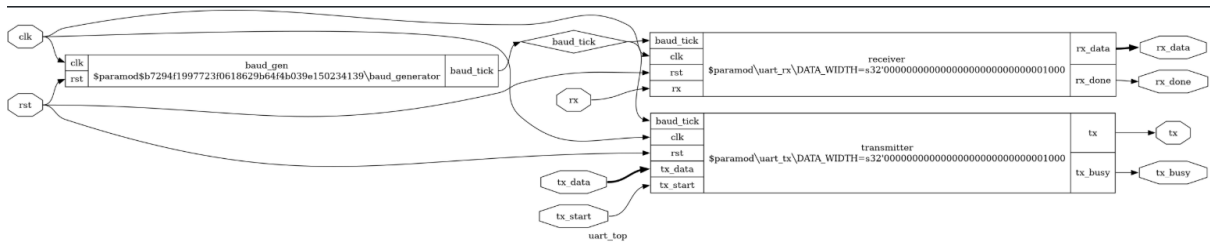


Figure 3.4: RTL hierarchy of the complete UART top module generated using Yosys.

3.6 Functional Verification

Each RTL module was verified independently using dedicated Verilog testbenches. After successful individual verification, the complete UART IP core was simulated to validate end-to-end serial communication.

The functional verification confirmed:

- Correct baud tick generation.
- Proper UART frame transmission.
- Accurate serial-to-parallel data reception.
- Successful integration of transmitter, receiver and baud generator.

The simulation waveforms are presented in Chapter ??, where the functional behavior of each module is analyzed in detail.

Chapter 4

RTL-to-GDSII Design Flow Using OpenLane2

This chapter describes the complete ASIC implementation flow used to transform the verified RTL design into a manufacturable GDSII layout. The implementation was performed using the open-source OpenLane2 flow, which integrates industry-standard tools for synthesis, physical design, routing, timing optimization, and physical verification.

The UART IP core was implemented using the SkyWater SKY130 open-source Process Design Kit (PDK). The design flow begins with synthesizable Verilog RTL and automatically progresses through logic synthesis, floorplanning, placement, clock tree synthesis, routing, and physical verification before generating the final GDSII layout.

4.1 OpenLane2 Design Flow

OpenLane2 provides an automated RTL-to-GDSII implementation flow by integrating multiple open-source EDA tools into a unified framework. The complete design flow used in this project is illustrated in Figure 4.1.

The major stages of the implementation flow are summarized below.

Table 4.1: Major stages of the OpenLane2 ASIC design flow

Stage	Description
RTL Design	Verilog HDL implementation
RTL Linting	Design rule and syntax checking
Logic Synthesis	RTL converted into gate-level netlist
Floorplanning	Core and I/O planning
Placement	Standard cell placement
Clock Tree Synthesis	Clock distribution network generation
Routing	Signal interconnection
Physical Verification	DRC and LVS verification
GDSII Generation	Final layout generation

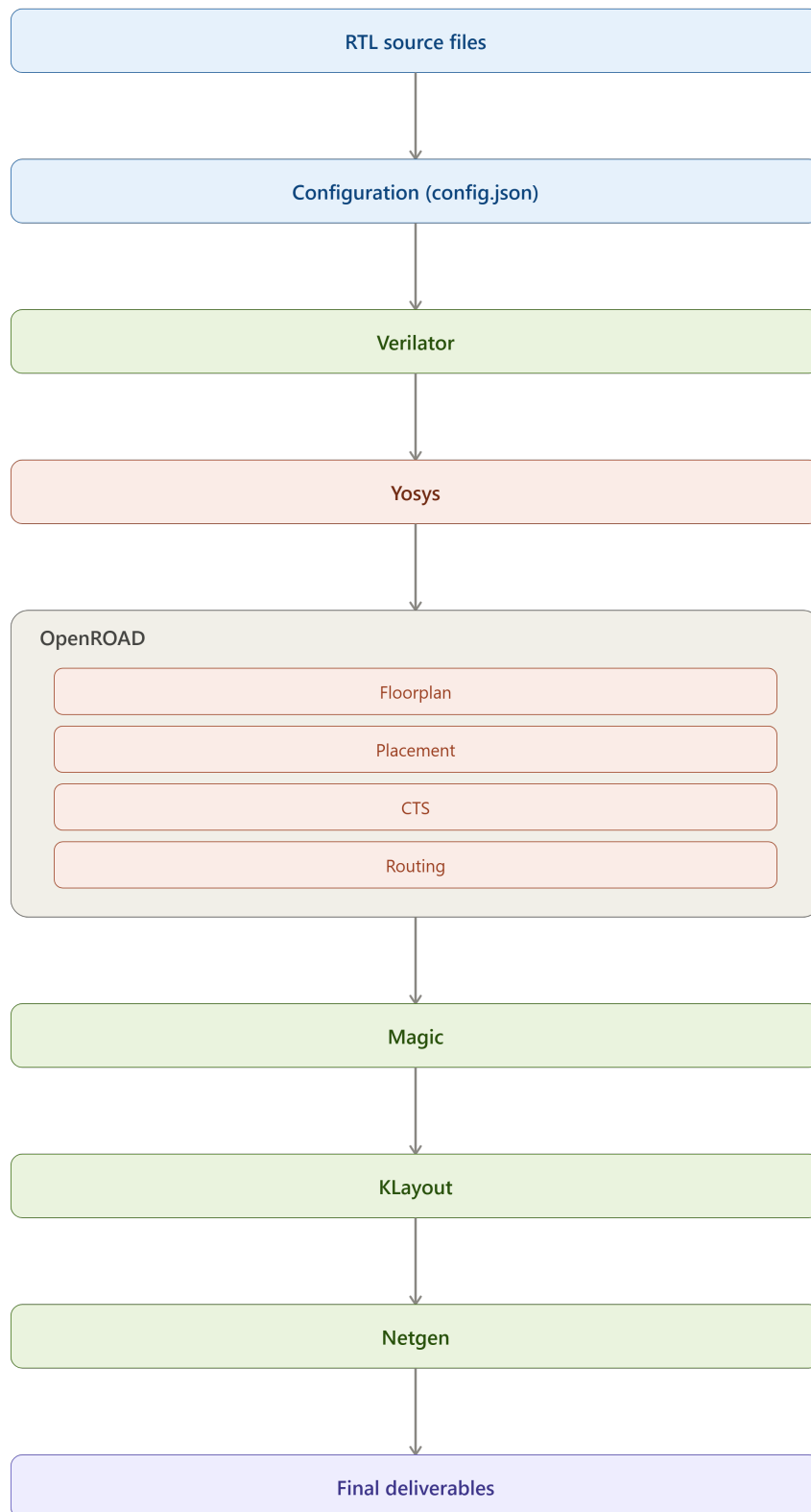


Figure 4.1: Complete RTL-to-GDSII implementation flow using OpenLane2.

4.2 Project Directory Structure

The UART project was organized into separate directories for RTL source files, testbenches, configuration files, simulation outputs, reports, and implementation results.

```
uart_asic_ip/  
|  
|-- rtl/  
|   |-- uart_top.v  
|   |-- uart_tx.v  
|   |-- uart_rx.v  
|   '-- baud_generator.v  
|  
|-- tb/  
|  
|-- waveforms/  
|  
|-- sim/  
|  
|-- config.json  
|  
|-- runs/  
|  
|-- reports/  
|  
|-- logs/  
|  
'-- results/
```

This directory organization simplifies project management and separates design sources from generated implementation files.

4.3 Design Configuration

OpenLane2 uses a `config.json` file to define the design-specific implementation parameters. This configuration specifies the top module, RTL source files, target PDK, clock information, and implementation constraints.

The primary configuration parameters used in this project are summarized in Table 4.2.

4.4 RTL Linting Using Verilator

Before synthesis, the RTL design was analyzed using Verilator to detect syntax errors, unsupported constructs, signal mismatches, and coding issues.

RTL linting improves design quality by identifying potential implementation problems before the synthesis stage. Successful completion of RTL linting ensures that the Verilog source files are suitable for logic synthesis.

Table 4.2: Important OpenLane2 configuration parameters

Parameter	Purpose
DESIGN_NAME	Top module name
VERILOG_FILES	RTL source files
CLOCK_PORT	Primary clock input
CLOCK_PERIOD	Target clock period
PDK	SkyWater SKY130 PDK
STD_CELL_LIBRARY	SKY130 standard cells

4.5 Logic Synthesis Using Yosys

Logic synthesis converts the behavioral Verilog description into a technology-mapped gate-level netlist using the SkyWater SKY130 standard cell library.

The synthesis process includes RTL parsing, hierarchy checking, logic optimization, technology mapping, and netlist generation.

Figure 4.2 illustrates the major synthesis stages.

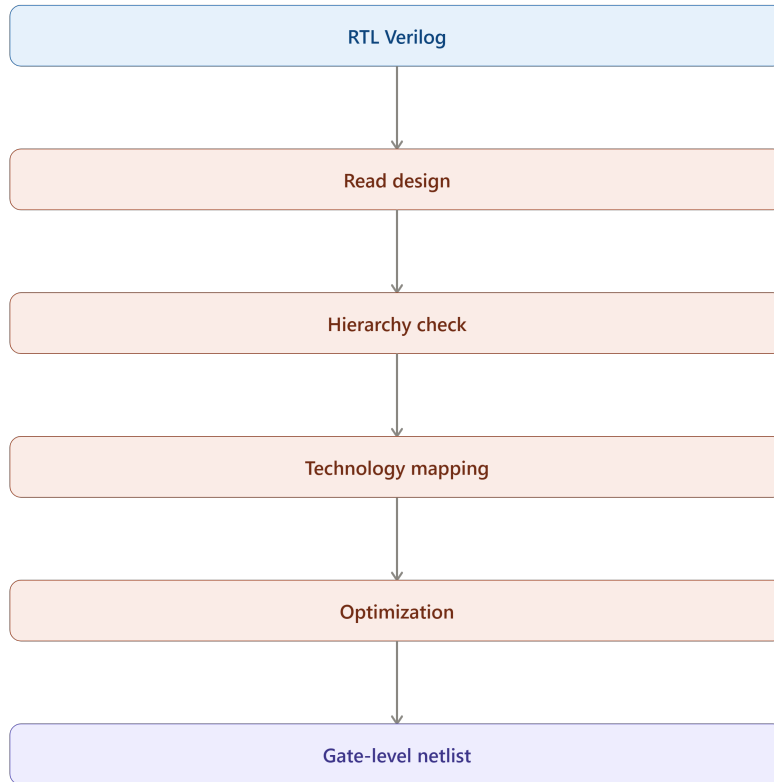


Figure 4.2: Logic synthesis flow performed using Yosys.

The synthesized gate-level netlist generated during this stage serves as the input for the subsequent physical design stages.

4.6 Complete OpenLane2 Implementation Pipeline

The overall implementation pipeline combines all open-source tools into a single automated ASIC design flow. Each tool performs a dedicated task and passes its output to the next stage until the final GDSII layout is produced.

Table 4.3 summarizes the primary EDA tools used throughout the implementation flow.

Table 4.3: Open-source EDA tools used in this project

Tool	Function
Verilator	RTL linting and design checking
Yosys	Logic synthesis
OpenROAD	Physical implementation
Magic	Layout generation and DRC
KLayout	Layout visualization and DRC
Netgen	Layout versus schematic (LVS) verification

The outputs generated during each stage of the implementation flow are summarized in Table 4.4.

Table 4.4: Generated outputs during RTL-to-GDSII implementation

Stage	Output
RTL Design	Verilog source files
Logic Synthesis	Gate-level netlist
Floorplanning	Floorplan DEF
Placement	Placed DEF
CTS	Clock tree
Routing	Routed DEF
Physical Verification	DRC and LVS reports
Final Output	GDSII layout

The detailed implementation of floorplanning, placement, clock tree synthesis, routing, and physical verification is presented in the following chapters.

Chapter 5

Physical Design Implementation

After logic synthesis, the gate-level netlist was imported into the OpenROAD engine through the OpenLane2 flow for physical implementation. This stage transforms the logical design into a manufacturable layout by determining standard cell locations, constructing the clock network, and routing signal interconnections. Using the SkyWater SKY130 PDK, OpenLane2 automatically performed floorplanning, placement, clock tree synthesis (CTS), global routing, and detailed routing.

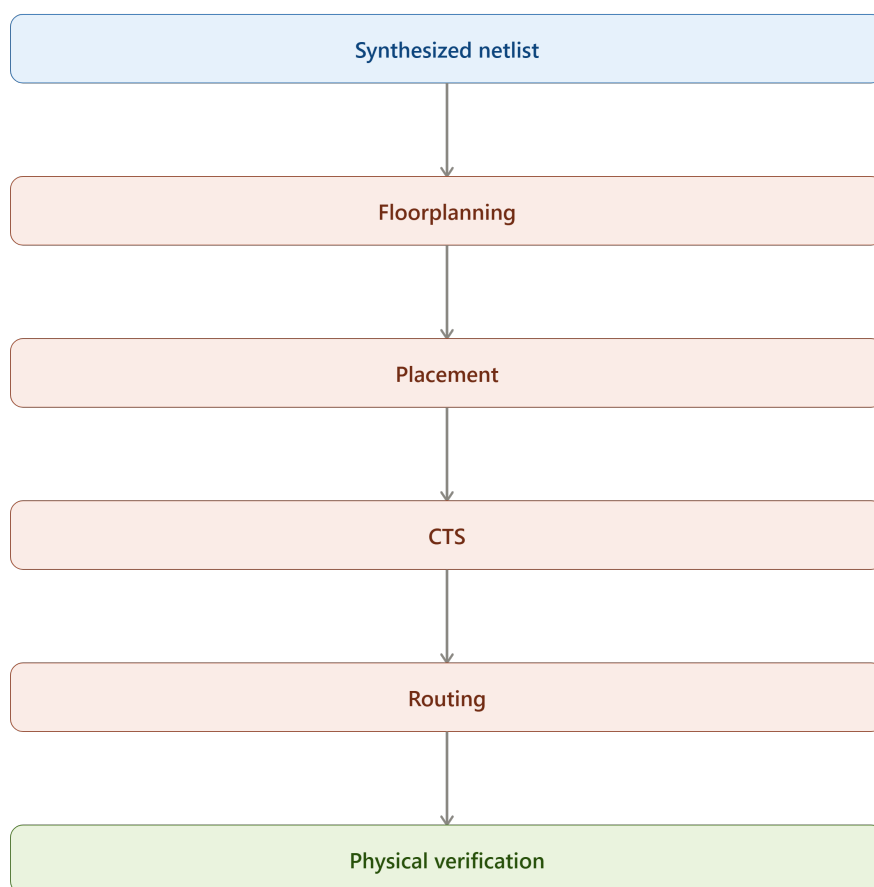


Figure 5.1: Physical design implementation flow using OpenROAD.

5.1 Floorplanning

Floorplanning is the first stage of physical implementation where the die area, core area, placement rows, utilization, and I/O pin locations are established. A well-planned floorplan provides sufficient routing resources and minimizes congestion during later stages of implementation.

Figure 5.2 shows the generated floorplan of the UART IP core obtained from OpenROAD.

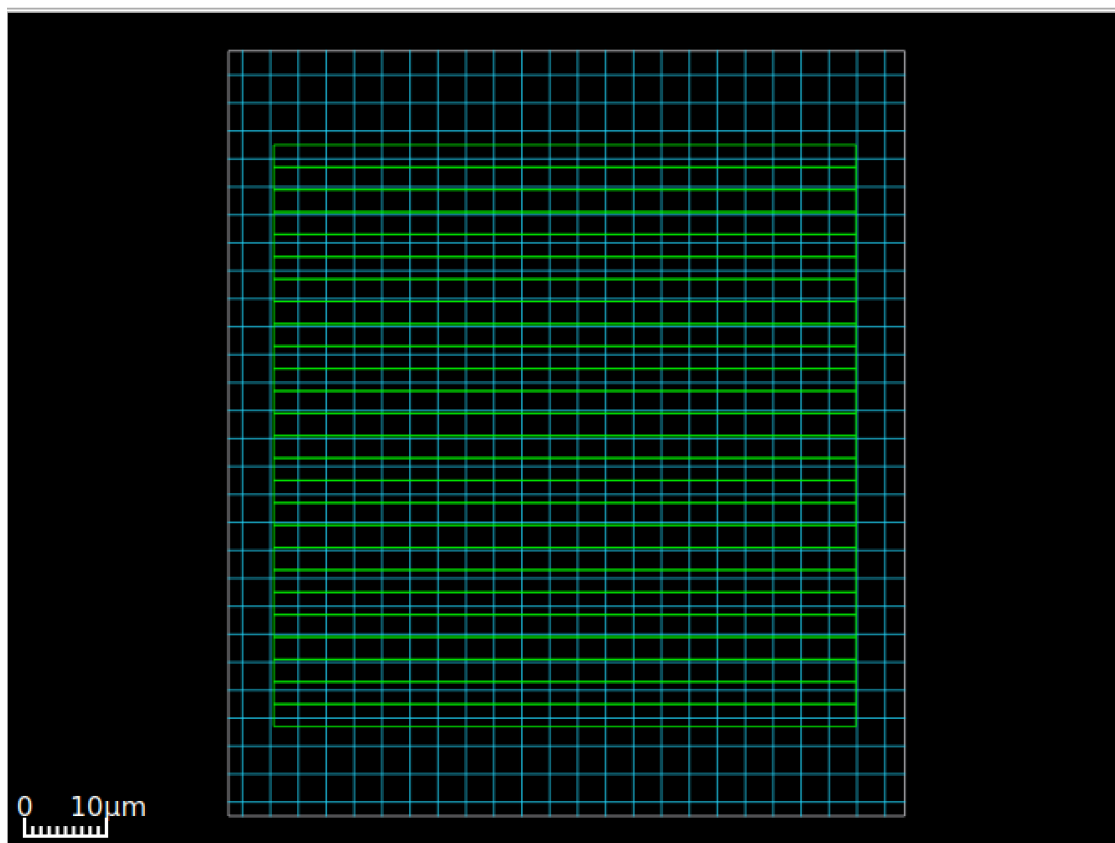


Figure 5.2: Generated floorplan of the UART ASIC design.

The floorplanning report generated during implementation is summarized in Table 5.1.

Table 5.1: Floorplanning metrics

Parameter	Value
Die Area	7658.18 μm^2
Core Area	5009.80 μm^2
Standard Cell Area	3210.58 μm^2
Instance Count	375
Core Utilization	64.09%
I/O Pins	25

The generated floorplan achieved a core utilization of approximately 64%, providing sufficient whitespace for placement optimization and routing while maintaining an efficient silicon area.

5.2 Global and Detailed Placement

After floorplanning, OpenROAD performs global placement to determine approximate locations of all standard cells while minimizing wirelength and improving timing. This is followed by detailed placement, where every standard cell is aligned to legal placement rows and overlap violations are removed.

Figure 5.3 shows the placed standard-cell layout generated during the placement stage.

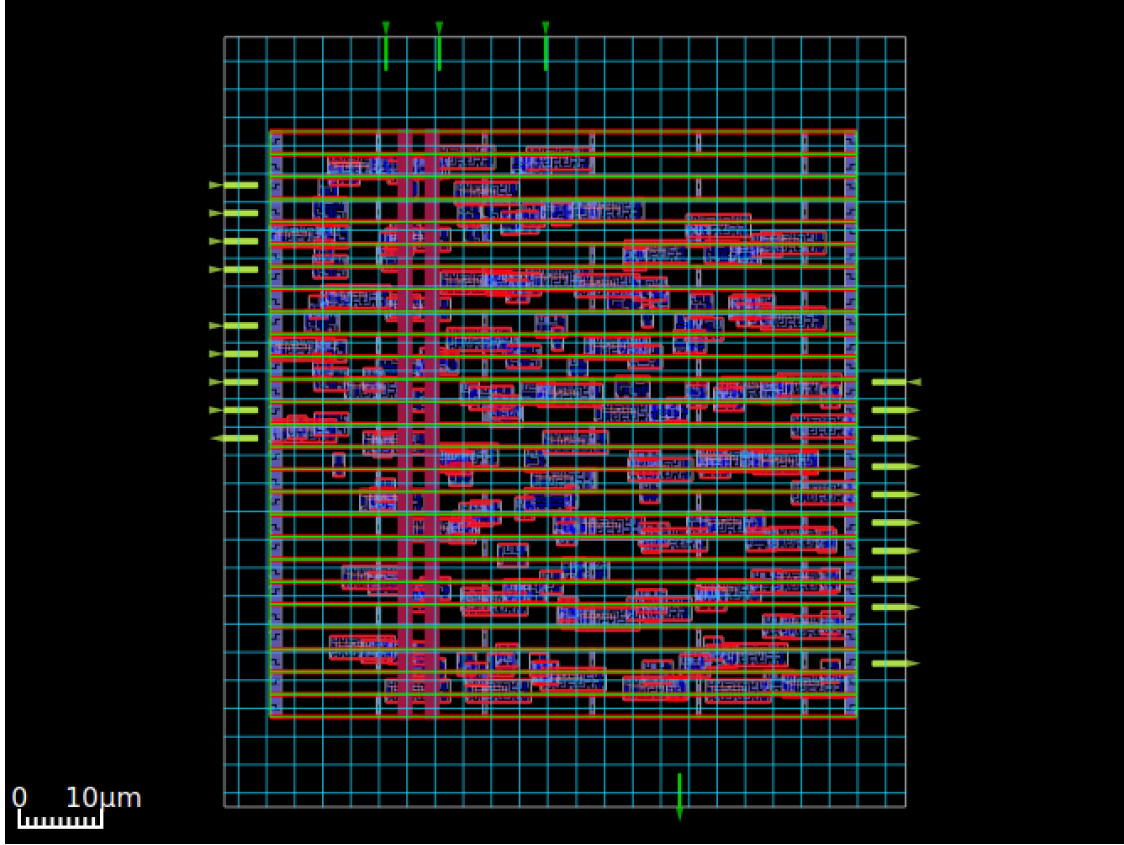


Figure 5.3: Standard-cell placement generated by OpenROAD.

The placement statistics obtained from the OpenLane2 implementation are summarized in Table 5.2.

Table 5.2: Placement metrics

Parameter	Value
Total Displacement	0.0 μm
Average Displacement	0.0 μm
Maximum Displacement	0.0 μm
HPWL Before Optimization	3351.5 μm
HPWL After Optimization	3207.2 μm
HPWL Improvement	4.3%
Standard Cell Utilization	51.65%

OpenROAD also inserted additional cells required for placement legalization and timing optimization. The resulting cell distribution is shown in Table 5.3.

Table 5.3: Cell distribution after detailed placement

Cell Type	Count
Sequential Cells	55
Combinational Cells	156
Timing Repair Buffers	31
Tap Cells	70
Fill Cells	52
Inverters	8
Total Cells	372

The detailed placement stage completed without overlap violations. The reported zero displacement indicates that all standard cells were successfully legalized while preserving the optimized placement generated during global placement. Furthermore, the reduction in Half-Perimeter Wire Length (HPWL) after optimization demonstrates improved routing efficiency and timing quality.

5.3 Clock Tree Synthesis

Clock Tree Synthesis (CTS) is performed after placement to distribute the clock signal uniformly to all sequential elements while minimizing clock skew, insertion delay, and timing violations. During this stage, OpenROAD inserts clock buffers and inverters to construct an optimized clock distribution network.

Figure 5.4 shows the CTS stage.

The CTS report generated during implementation is summarized in Table 5.4.

Table 5.4: Clock Tree Synthesis statistics

Parameter	Value
Clock Nets	6
Clock Buffers Inserted	5
Clock Inverters Inserted	3
Timing Repair Buffers	78
Sequential Cells	55
Total Cell Instances	427
Total Cell Area	3405.77 μm^2
Core Utilization	64.09%

The CTS stage successfully generated the clock distribution network without introducing clock-related violations. The insertion of dedicated clock buffers and inverters balanced the clock paths while maintaining acceptable utilization. Additional timing repair buffers were inserted automatically by OpenROAD to improve timing performance before routing.

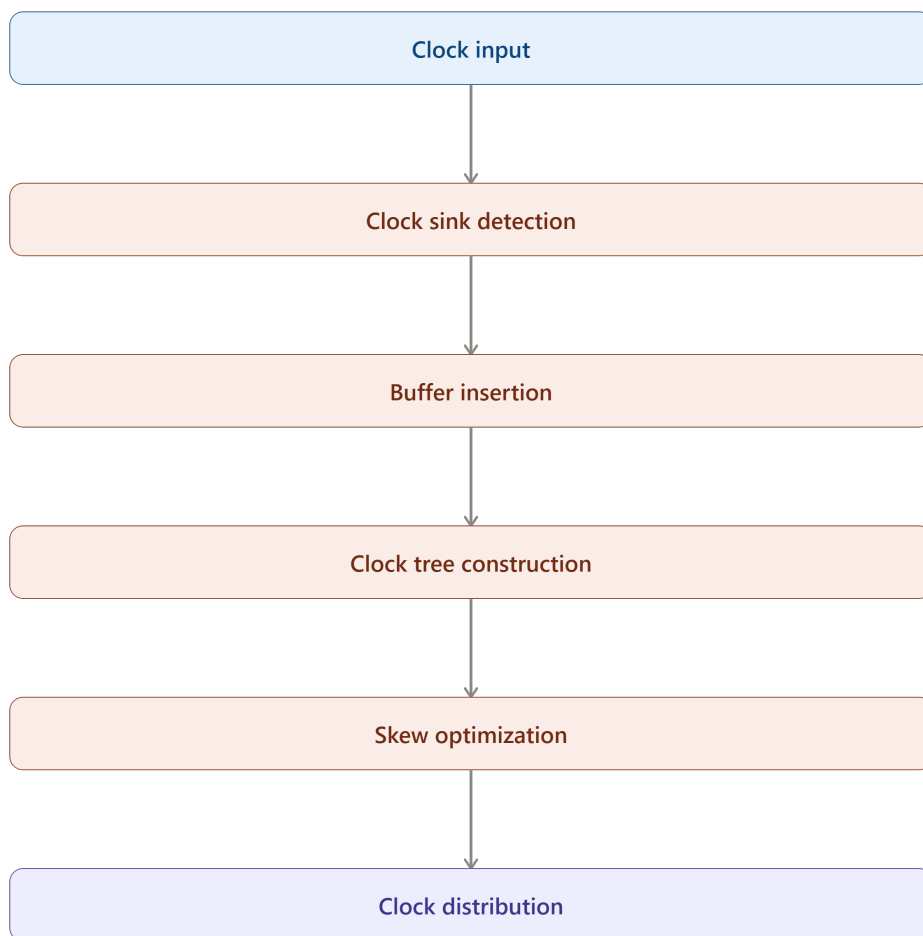


Figure 5.4: Clock Tree Synthesis (CTS) stages.

5.4 Global and Detailed Routing

Following Clock Tree Synthesis (CTS), OpenROAD performs routing to establish all electrical connections between the placed standard cells. The routing process consists of two stages: global routing, which determines approximate routing paths, and detailed routing, which generates the final metal interconnections while satisfying the SKY130 design rules.

Figure 5.5 shows the routed layout generated after the completion of detailed routing.

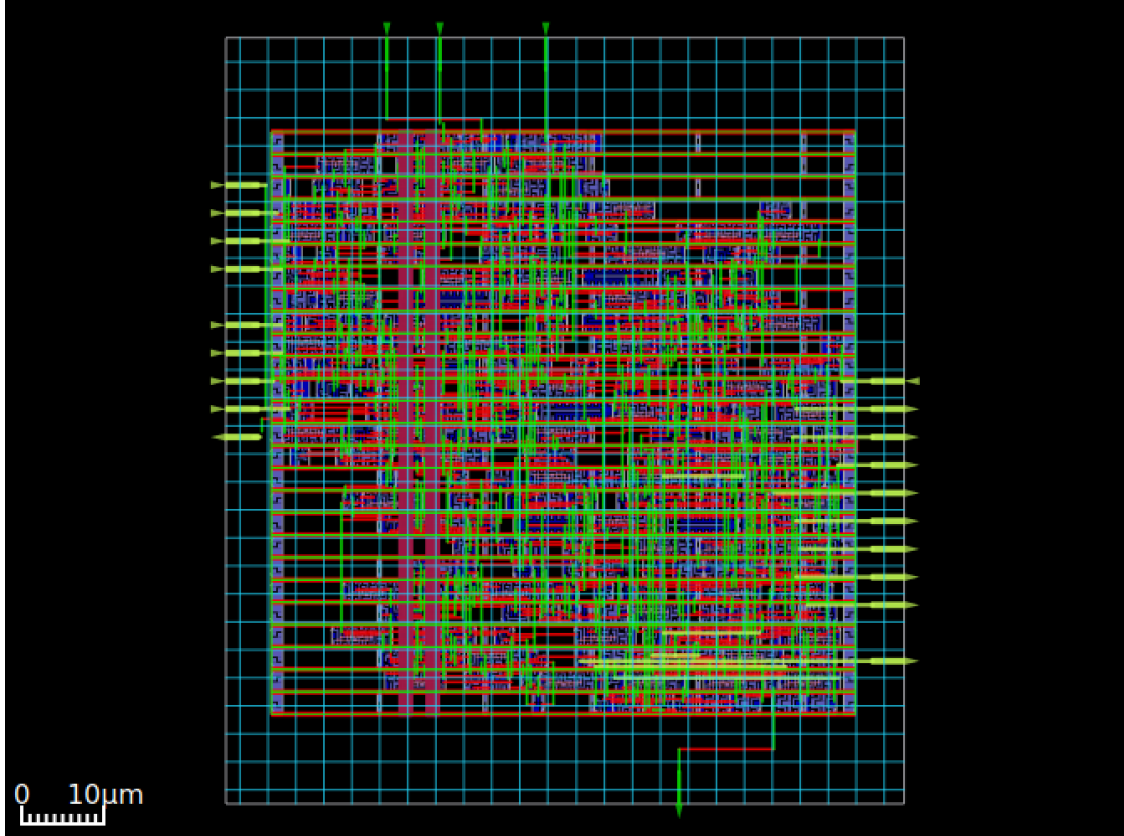


Figure 5.5: Final routed layout generated by OpenROAD.

The routing stage completed successfully without any congestion or overflow. Table 5.5 summarizes the routing statistics obtained from the OpenLane2 implementation.

Table 5.5: Global and detailed routing statistics

Parameter	Value
Routed Nets	314
Total Wirelength	8404 μm
Total Vias	1804
Routing Resource Usage	14.71%
Routing Overflow	0
Antenna Violating Nets	0
Antenna Violating Pins	0

The routing resource utilization for each metal layer is summarized in Table 5.6.

Table 5.6: Routing resource utilization by metal layer

Layer	Resources	Demand	Usage
Metal1	1406	330	23.47%
Metal2	1428	311	21.78%
Metal3	940	17	1.81%
Metal4	558	0	0.00%
Metal5	140	0	0.00%

The routing results demonstrate efficient utilization of the available routing resources. The absence of routing overflow confirms that all signal nets were successfully connected without congestion. Furthermore, no antenna violations were reported, indicating that the routing satisfies the manufacturing requirements of the SKY130 process.

5.5 Physical Design Summary

Table 5.7 summarizes the key results obtained throughout the physical design implementation.

Table 5.7: Summary of physical design implementation

Metric	Value
Design Instances	375
Core Area	5009.80 μm^2
Die Area	7658.18 μm^2
Core Utilization	64.09%
Clock Nets	6
Clock Buffers	5
Clock Inverters	3
Timing Repair Buffers	78
Routed Nets	314
Wirelength	8404 μm
Total Vias	1804
Routing Overflow	0
Antenna Violations	0

The physical implementation of the UART IP core was successfully completed using the OpenLane2 flow. All implementation stages, including floorplanning, placement, clock tree synthesis, and routing, were completed without critical issues. The generated routed layout serves as the input for the physical verification and signoff process presented in the next chapter.

Chapter 6

Physical Verification and Signoff

After the completion of physical implementation, the UART layout was subjected to a comprehensive verification process to ensure that the generated design satisfies timing, power integrity, layout correctness, and manufacturability requirements. The verification stages were performed using the integrated tools available in the OpenLane2 flow, including OpenRCX, OpenSTA, Magic, Netgen, and the manufacturability checker.

6.1 Verification Flow

The verification stages performed after routing are illustrated in Figure 6.1.

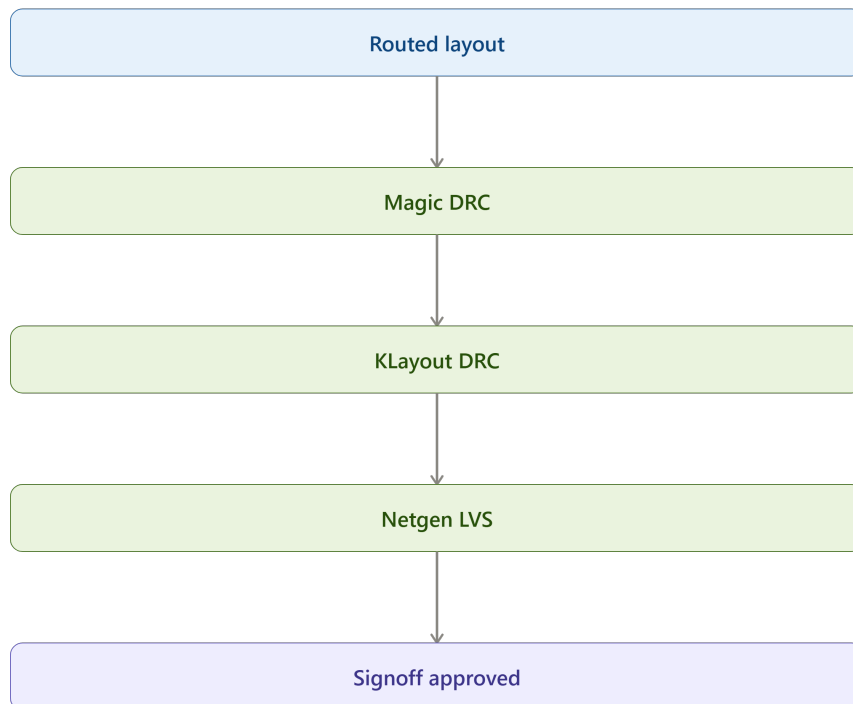


Figure 6.1: Physical verification and signoff flow.

6.2 RC Extraction

After routing, parasitic resistance and capacitance associated with the interconnects were extracted using OpenRCX. The extracted parasitic information was stored in Standard Parasitic Exchange Format (SPEF) files and used for accurate post-layout timing analysis.

The RC extraction stage generated parasitic information for multiple process corners, as summarized in Table 6.1.

Table 6.1: Generated SPEF files

Operating Corner	Generated File
Minimum	uart_top.min.spef
Nominal	uart_top.nom.spef
Maximum	uart_top.max.spef

The successful generation of all three SPEF files confirms that parasitic extraction was completed successfully without any extraction errors.

6.3 Static Timing Analysis

Static Timing Analysis (STA) was performed using the extracted parasitic information to verify timing performance under multiple operating corners.

The timing analysis evaluated setup and hold constraints across different process, voltage, and temperature (PVT) conditions. The analysis completed successfully without reporting critical timing violations, indicating successful timing closure after routing.

6.4 IR Drop Analysis

Power integrity analysis was performed to evaluate voltage drops across the power distribution network. The analysis confirmed that both the power (VPWR) and ground (VGND) networks maintain stable operating voltages throughout the design.

The measured voltage drops are summarized in Table 6.2.

Table 6.2: IR-drop analysis results

Parameter	VPWR	VGND
Worst Voltage	1.79984 V	0.000233 V
Average IR Drop	0.0000416 V	0.0000400 V
Worst IR Drop	0.0001605 V	0.0002330 V
Percentage Drop	0.01%	0.01%

The observed voltage drop is extremely small, indicating a robust power distribution network suitable for reliable circuit operation.

6.5 Design Rule Checking

Magic was used to perform Design Rule Checking (DRC) on the routed layout. The generated layout was verified against the SKY130 manufacturing design rules.

The verification completed successfully with zero DRC violations.

Table 6.3: Magic DRC summary

Parameter	Result
DRC Violations	0
Verification Status	PASS

6.6 Layout Versus Schematic Verification

Netgen was used to perform Layout Versus Schematic (LVS) verification. This step compares the extracted layout netlist against the synthesized gate-level netlist to ensure logical equivalence.

The LVS verification completed successfully, confirming that the physical layout matches the intended circuit implementation.

Table 6.4: LVS verification summary

Parameter	Result
Layout Matches Schematic	PASS
Verification Status	PASS

6.7 Final GDSII Layout

The verified UART layout was exported in GDSII format after successful completion of all implementation and verification stages. Figure 6.2 shows the final layout viewed in KLayout.

6.8 Manufacturability Assessment

The final manufacturability report summarizes the overall fabrication readiness of the design by combining the results of antenna checking, DRC, and LVS verification.

The verification summary is presented in Table 6.5.

The successful completion of all verification stages confirms that the UART ASIC implementation satisfies the required design rules and verification checks. The generated layout is therefore considered suitable for fabrication using the SKY130 process.

Chapter 7

Results and Discussion

This chapter presents the key outcomes obtained during the complete RTL-to-GDSII implementation of the UART IP core. The results include functional verification, synthesis statistics, physical implementation metrics, routing performance, power integrity analysis, physical verification, and the final GDSII layout.

7.1 Functional Verification

The UART IP core was functionally verified using dedicated Verilog testbenches. The generated simulation waveforms confirm the correct operation of each RTL module before physical implementation.

7.1.1 Baud Rate Generator

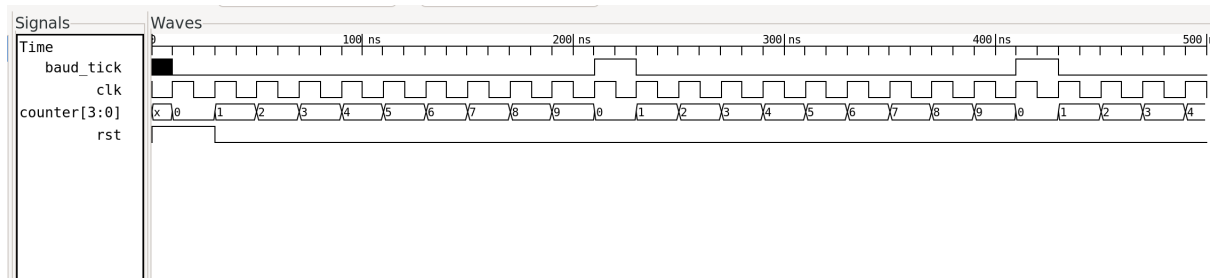


Figure 7.1: Simulation waveform of the Baud Rate Generator.

The waveform verifies that the baud generator correctly divides the system clock and produces periodic baud timing pulses for UART communication.

7.1.2 UART Transmitter

The transmitter correctly generates the UART frame consisting of one start bit, eight data bits, and one stop bit.

7.1.3 UART Receiver

The receiver successfully reconstructs the transmitted serial data and asserts the receive-complete signal after reception.

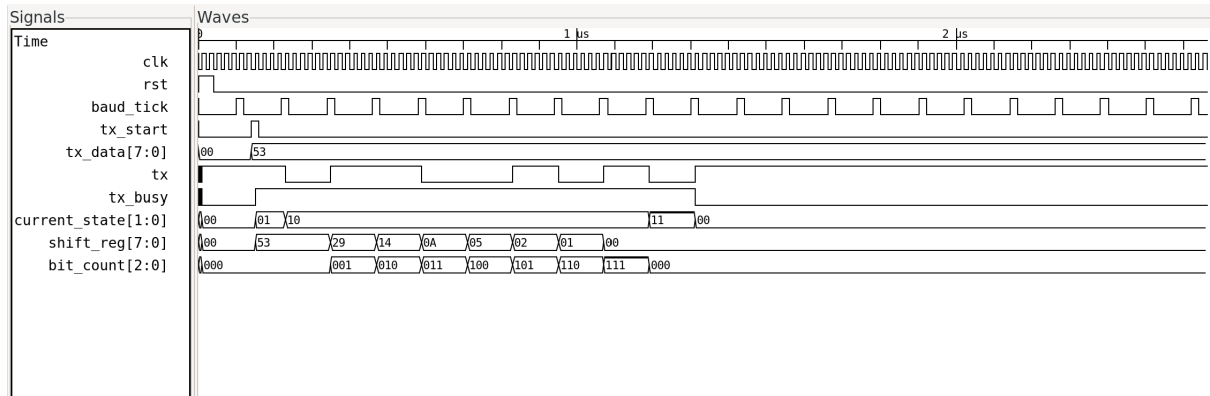


Figure 7.2: Simulation waveform of the UART transmitter.

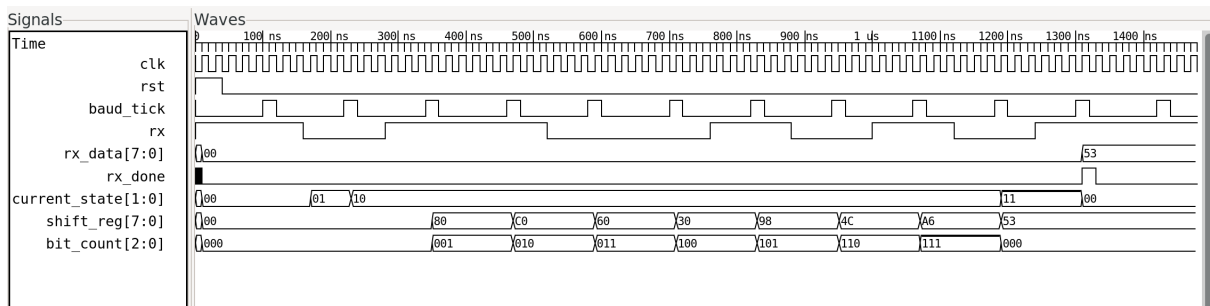


Figure 7.3: Simulation waveform of the UART receiver.

7.1.4 UART Top Module

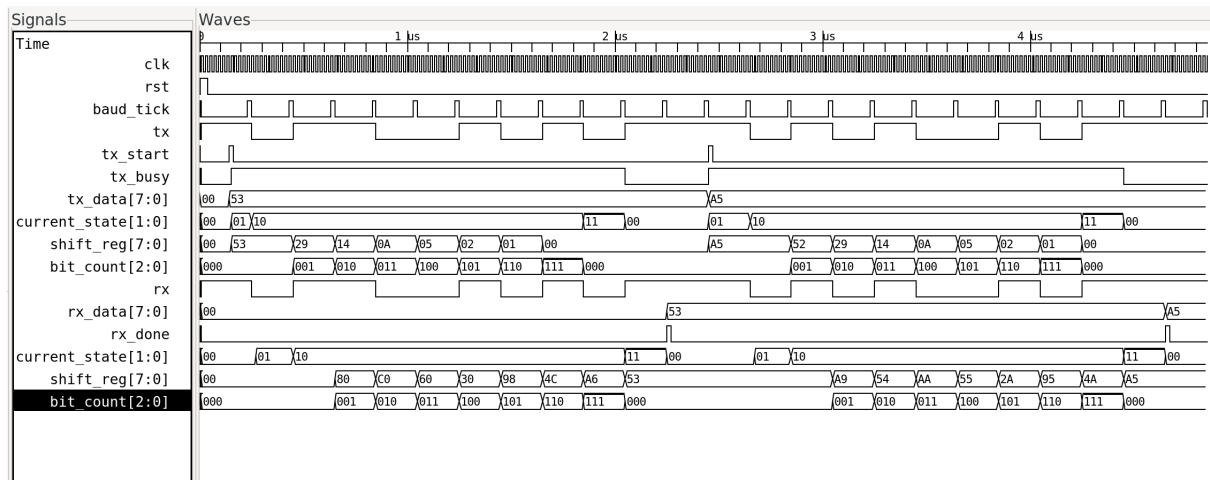


Figure 7.4: Simulation waveform of the complete UART top module.

The complete UART system performs successful end-to-end serial communication, confirming proper integration of all RTL modules.

7.2 RTL Synthesis Results

The RTL was synthesized using Yosys and mapped to the SKY130 standard-cell library.

Table 7.1: RTL synthesis summary

Parameter	Value
Standard Cells	219
Sequential Cells	55
Cell Area	2539.936 μm^2
Sequential Area Ratio	46.06%
Technology Library	SKY130 HD

The synthesis completed successfully and generated a technology-mapped gate-level netlist suitable for ASIC implementation.

7.3 Floorplanning Results

Table 7.2: Floorplanning results

Metric	Value
Die Area	7658.18 μm^2
Core Area	5009.80 μm^2
Standard Cell Area	3210.58 μm^2
Core Utilization	64.09%
Design Instances	375
I/O Pins	25

The generated floorplan provides sufficient whitespace for routing while maintaining efficient silicon utilization.

7.4 Placement Results

Table 7.3: Placement statistics

Metric	Value
HPWL Before Optimization	3351.5 μm
HPWL After Optimization	3207.2 μm
HPWL Improvement	4.3%
Total Displacement	0 μm
Average Displacement	0 μm
Maximum Displacement	0 μm

The placement stage successfully legalized all cells while reducing the estimated routing length.

7.5 Clock Tree Synthesis Results

Table 7.4: CTS statistics

Metric	Value
Clock Nets	6
Clock Buffers	5
Clock Inverters	3
Timing Repair Buffers	78
Total Cell Area	3405.77 μm^2

The generated clock tree provides balanced clock distribution with automatic timing optimization.

7.6 Routing Results

Table 7.5: Routing statistics

Metric	Value
Routed Nets	314
Wirelength	8404 μm
Total Vias	1804
Routing Utilization	14.71%
Routing Overflow	0
Antenna Violations	0

The routing stage completed successfully with zero congestion overflow and no antenna violations.

7.7 Power Integrity Results

Table 7.6: IR-drop summary

Parameter	VPWR	VGND
Worst Voltage	1.79984 V	0.000233 V
Worst IR Drop	0.0001605 V	0.0002330 V
Average IR Drop	0.0000416 V	0.0000400 V
Percentage Drop	0.01%	0.01%

The measured voltage drop is extremely small, indicating a robust power delivery network.

7.8 Physical Verification Results

Table 7.7: Verification summary

Verification	Status
Magic DRC	PASS
DRC Violations	0
Netgen LVS	PASS
Antenna Check	PASS
Manufacturability	PASS

All physical verification stages completed successfully, confirming that the generated layout satisfies the SKY130 design rules.

7.9 Overall Discussion

The UART IP core was successfully implemented using the complete open-source RTL-to-GDSII design flow. Functional verification confirmed correct UART operation, while physical implementation achieved efficient utilization, successful timing closure, zero routing overflow, negligible IR drop, and clean physical verification results. The final layout passed DRC, LVS, antenna, and manufacturability verification, demonstrating that the generated GDSII layout is suitable for fabrication using the SkyWater SKY130 PDK.

Chapter 8

Conclusion

This project successfully demonstrated the complete RTL-to-GDSII implementation of a UART (Universal Asynchronous Receiver-Transmitter) IP core using the open-source OpenLane2 ASIC design flow and the SkyWater SKY130 Process Design Kit (PDK). The UART was designed in Verilog HDL using a modular architecture consisting of a Baud Rate Generator, UART Transmitter, UART Receiver, and a top-level integration module. Functional verification through simulation confirmed the correct operation of each module as well as the complete UART system.

The verified RTL was synthesized using Yosys and subsequently implemented through the OpenROAD physical design flow. The implementation included floorplanning, standard-cell placement, clock tree synthesis, global routing, and detailed routing, resulting in a complete physical layout of the UART IP core. Throughout the implementation process, the design achieved successful placement, balanced clock distribution, congestion-free routing, and efficient utilization of the available routing resources.

Post-layout verification was performed using OpenRCX, OpenSTA, Magic, and Netgen. RC extraction, timing analysis, IR-drop analysis, Design Rule Checking (DRC), Layout Versus Schematic (LVS) verification, and manufacturability assessment were all completed successfully. The generated layout reported zero routing overflow, zero antenna violations, zero DRC violations, successful LVS matching, and negligible IR drop, demonstrating the correctness and fabrication readiness of the design.

Overall, this project provides practical experience with the complete digital ASIC implementation flow using entirely open-source EDA tools. It demonstrates the transformation of a synthesizable RTL design into a verified GDSII layout while exposing every major stage of modern ASIC design, including synthesis, physical implementation, verification, and signoff.

Future Work

The current UART IP core can be extended to support additional communication features and more complex System-on-Chip (SoC) designs. Possible future enhancements include:

- Configurable baud rates and programmable data formats.
- Support for parity generation and parity error detection.
- FIFO-based transmit and receive buffers for improved throughput.

- Interrupt-driven UART operation for processor-based systems.
- Integration with standard on-chip buses such as APB or AXI-Lite.
- FPGA prototyping followed by fabrication through an MPW shuttle run.

Bibliography

- [1] *OpenLane2 Documentation*, The OpenLane Project, 2025. [Online]. Available: <https://openlane2.readthedocs.io/>
- [2] *OpenROAD Project Documentation*, The OpenROAD Project, 2025. [Online]. Available: <https://openroad.readthedocs.io/>
- [3] *Yosys Open SYNthesis Suite Documentation*, YosysHQ, 2025. [Online]. Available: <https://yosyshq.readthedocs.io/>
- [4] *Verilator User Guide*, Verilator Project, 2025. [Online]. Available: <https://verilator.org/guide/latest/>
- [5] *Magic VLSI Layout Tool Documentation*, Magic VLSI, 2025. [Online]. Available: <http://opencircuitdesign.com/magic/>
- [6] *KLayout Layout Viewer and Editor Documentation*, KLayout Project, 2025. [Online]. Available: <https://www.klayout.de/doc.html>
- [7] *Netgen LVS Documentation*, Open Circuit Design, 2025. [Online]. Available: <http://opencircuitdesign.com/netgen/>
- [8] *SkyWater SKY130 Process Design Kit Documentation*, SkyWater Technology and Google, 2025. [Online]. Available: <https://skywater-pdk.readthedocs.io/>
- [9] *OpenRCX Documentation*, The OpenROAD Project, 2025. [Online]. Available: <https://openroad.readthedocs.io/>
- [10] *OpenSTA Static Timing Analyzer Documentation*, The OpenROAD Project, 2025. [Online]. Available: <https://github.com/The-OpenROAD-Project/OpenSTA>
- [11] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall, 2003.
- [12] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Addison-Wesley, 2011.
- [13] M. M. Mano and M. D. Ciletti, *Digital Design*, 6th ed. Pearson, 2018.
- [14] J. Bhasker, *A Verilog HDL Primer*, 3rd ed. Star Galaxy Publishing, 2007.
- [15] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed. McGraw-Hill, 2014.
- [16] J. Axelson, *Serial Port Complete*, 2nd ed. Lakeview Research, 2007.

- [17] *Icarus Verilog Documentation*, Icarus Verilog Project, 2025. [Online]. Available: <https://steveicarus.github.io/iverilog/>
- [18] *GTKWave Waveform Viewer*, GTKWave Project, 2025. [Online]. Available: <https://gtkwave.sourceforge.net/>